SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

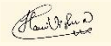
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	HARI VISHNU N	**Reg. No.:**	192524319
Section / Batch:	SLOT D / 2025	**Date:**	08-10-2026

Code of Conduct

I HARI VISHNU N / 192524319 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	**Q2**
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	**Q1**
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	**Q1**
4. The Nature of Environments	Task-environment properties and their classification	**Q1**
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	**Q1**
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	**Q2**
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	**Q2, Q4**
8. Searching for Solutions	Search graph, frontier, explored set, node expansion	**Q3**
9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-cost search	**Q3**
10. Search with Partial Information	Sensorless problems, belief states, percept update	**Q4**

Annexure A – Dry Run Evaluation

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent. **[4 Marks]**

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each. **[4 Marks]**

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step). **[9 Marks]**

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1					
2					
3					
4					
5					
6					

(d) Compute the total performance of X and of Y over the 6 steps. **[3 Marks]**

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent. **[5 Marks]**

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes

Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

(a) Complete the table. **[8 Marks]**

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)		
General Problem Solver (Newell and Simon, 1957)		
Aristotle's syllogisms and formal logic		
An agent that does the right thing to maximise expected performance		

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost. **[6 Marks]**

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged). **[3 Marks]**

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost. **[3 Marks]**

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts. **[5 Marks]**

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

Edge	Cost	Edge	Cost
S – A	1	B – D	1
S – B	4	C – G	6
A – C	2	D – G	2
A – D	5		

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue. **[6 Marks]**

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1			
2			
3			
4			
5			
6			

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left. **[5 Marks]**

Iteration	Node Popped	Stack after Pushing Children
1		
2		
3		
4		

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found. **[7 Marks]**

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1			
2			
3			
4			

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
5			
6			

(d) Complete the comparison table. **[4 Marks]**

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS				
DFS				
UCS				

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example). **[3 Marks]**

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost. **[3 Marks]**

(b) Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action. **[6 Marks]**

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	
2	Suck	
3	Left	

Step	Action	Belief State after Action
4	Suck	

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update). **[8 Marks]**

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	
1	Suck		[Left, Clean]	
2	Right		[Right, Dirty]	

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states? **[3 Marks]**

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational. **[5 Marks]**

ANSWERS:

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

(a) PEAS Description:

PEAS Element	Description
Performance measure	+10 for every square cleaned (Suck on a Dirty square); −1 for every Left or Right move; total score taken over T = 6 time steps.
Environment	Two squares, A and B, each Clean or Dirty; the agent starts in A; both squares initially Dirty; dirt never reappears.
Actuators	Wheels / motor (Left, Right), suction unit (Suck), and NoOp (do nothing).
Sensors	Location sensor (A or B) and dirt sensor for the current square only (Clean / Dirty).

Justification: PEAS defines the task environment completely. The performance measure rewards cleaning and penalises movement, so it is stated in terms of what is wanted in the environment, not how the agent behaves.

(b) Classification of the Task Environment:

Property	Classification	Justification
Observability	Partially observable	The agent senses only its own square's status, not the other square's.
Number of agents	Single agent	Only the vacuum agent acts; nothing else affects the world.
Determinism	Deterministic	The next state is fully fixed by the current state and action (Suck always cleans).
Episodic / Sequential	Sequential	Present actions (moving, sucking) change later percepts and the final score.
Static / Dynamic	Static	Dirt never reappears, so the world does not change while the agent decides.
Discrete / Continuous	Discrete	Finite squares, percepts, actions and time steps.

(c) Dry Run of Agent X and Agent Y (T = 6):

Assumptions: (i) the agent starts in A and both squares are Dirty; (ii) a Suck on a Dirty square makes it Clean; (iii) a move changes only the agent's location; (iv) Y updates its internal state with the current percept first and then chooses an action, and a square is added to the known-clean set once Y has seen it Clean or sucked it; (v) in the "Y: Known-Clean Squares" column the set shown is the state after the step.

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

Justification: Step 1: both agents see [A, Dirty], so both Suck (Y adds A to its known-clean set).

Step 2: A is Clean; X moves because its rule says "otherwise move". Y knows only A is clean, so B is unknown and it also moves Right.

Step 3: B is Dirty, so both Suck; Y now knows {A, B}.

Step 4: both see [B, Clean]. X still moves to the other square, but Y's model says both squares are clean, so it does NoOp.

Steps 5–6: X keeps shuttling between the squares; Y stays in B and does NoOp.

(d) Total Performance:

Step	1	2	3	4	5	6	Total
Agent X	+10	−1	+10	−1	−1	−1	**16**
Agent Y	+10	−1	+10	0	0	0	**19**

Agent X: $2 \times (+10) + 4$ moves $\times (-1) = 20 - 4 =$ **16**.

Agent Y: $2 \times (+10) + 1$ move $\times (-1) = 20 - 1 =$ **19**.

Justification: Both agents clean two squares (+20). X wastes four moves (steps 2, 4, 5, 6) but Y wastes only one (step 2, which was necessary to discover B's status). Hence Y scores 3 points higher.

(e) Interpretation:

Which agent is rational? Agent Y is rational. A rational agent selects, for each percept sequence, the action expected to maximise the performance measure using its percepts and built-in knowledge. Y scores 19 against X's 16, and every one of Y's actions is justified: it moved at step 2 because B's status was unknown and stopped moving once both squares were known clean. Agent X is not rational for this performance measure because it keeps paying −1 for moves that cannot gain anything, since dirt never reappears. (X would be a reasonable design only if dirt could reappear and patrolling were useful.)

Why do they act differently on the same percept [B, Clean] at step 4? The two agent programs have different structures. Agent X is a simple reflex agent: it uses condition–action rules on the current percept alone, so [B, Clean] always triggers "move to the other square" (Left). Agent Y is a model-based reflex agent: it keeps an internal state that records history (A was seen clean at steps 1–2, B at steps 3–4). Its decision depends on the percept and on this stored state, so with both squares known clean, it chooses NoOp. Identical percepts therefore give different actions because only Y remembers the past.

Rational vs omniscient: An omniscient agent knows the actual outcome of its actions and the whole state of the environment in advance, which is impossible in practice. A rational agent maximises expected performance given what it has perceived and knows. Y did not know at step 1 that B was dirty, so it had to go and look. That is rational even though, with hindsight, an omniscient agent would also have chosen Right. Rationality is therefore about expected, not actual, success, and does not require perfection.

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation:

Part A (a) Classification of Systems / Ideas:

System / Idea	Approach to AI	Era / Milestone	Justification
Turing Test proposed (1950)	Acting humanly	Gestation of AI (1943–1955)	Intelligence is judged by behaviour that a human interrogator cannot tell apart from a human's.
General Problem Solver (Newell and Simon, 1957)	Thinking humanly	Early enthusiasm, great expectations (1952–1969)	GPS compared its reasoning steps with the thinking steps of human subjects, a cognitive-modelling aim.
Aristotle's syllogisms and formal logic	Thinking rationally	Foundations – philosophy (before 1943)	These are "laws of thought": patterns of correct reasoning that always give correct conclusions from correct premises.

An agent that does the right thing to maximise expected performance	Acting rationally	Emergence of intelligent agents (1995–present)	This is the rational-agent view: success is measured by achieving the best expected outcome.

Part B (b) Problem Formulation – Water Jug Problem:

Component	Formulation
State representation	(x, y) where x = litres in jug X ($0 \le x \le 4$) and y = litres in jug Y ($0 \le y \le 3$).
Initial state	(0, 0): both jugs empty.
Actions	Fill X; Fill Y; Empty X; Empty Y; Pour X→Y; Pour Y→X.
Transition model	Fill X: (x, y) → (4, y). Fill Y: (x, y) → (x, 3). Empty X: (x, y) → (0, y). Empty Y: (x, y) → (x, 0). Pour X→Y: t = min(x, 3 − y), giving (x − t, y + t). Pour Y→X: t = min(y, 4 − x), giving (x + t, y − t).
Goal test	x = 2, i.e. the state is (2, y) for any y.
Path cost	Each action costs 1; the path cost is the number of actions.

Justification: The pour rule uses min() so that pouring stops when the source is empty or the target is full, exactly as stated in the problem. Only the amounts in the two jugs matter for solving the problem, so (x, y) is an adequate abstraction.

(c) Successor States:

State	Action	Successor
(0, 0)	Fill X	(4, 0)
(0, 0)	Fill Y	(0, 3)
(4, 0)	Fill Y	(4, 3)
(4, 0)	Empty X	(0, 0)
(4, 0)	Pour X→Y (t = min(4, 3) = 3)	(1, 3)

Justification: From (0, 0) the Empty and Pour actions leave the state unchanged, so there are 2 successors: {(4, 0), (0, 3)}. From (4, 0) Fill X, Empty Y and Pour Y→X are unchanged, so there are 3 successors: {(4, 3), (0, 0), (1, 3)}.

(d) State-Space Size and Shortest Solution:

Number of states: x has 5 values (0–4) and y has 4 values (0–3), so the state space contains $5 \times 4 = 20$ states. Of these only 14 are reachable from (0, 0): the six "interior" states (1,1), (1,2), (2,1), (2,2), (3,1) and (3,2) can never occur, because after any action at least one jug is empty or full.

Shortest solution (breadth-first levels): Level 0: (0,0). Level 1: (4,0), (0,3). Level 2: (4,3), (1,3), (3,0). Level 3: (1,0), (3,3). Level 4: (0,1), (4,2). Level 5: (4,1), (0,2). Level 6: (2,3) and (2,0), the goal. The first goal appears at level 6, so no solution is shorter.

Step	Action	State
0	Initial	(0, 0)
1	Fill Y	(0, 3)
2	Pour Y→X	(3, 0)
3	Fill Y	(3, 3)
4	Pour Y→X (X takes only 1 litre)	(4, 2)
5	Empty X	(0, 2)
6	Pour Y→X	(2, 0) ✓ Goal

Solution: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0). Path cost = 6.

Justification: An equally short alternative is (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3), also of cost 6. Because every action costs 1 and BFS reaches the goal first at level 6, 6 is the optimal cost.

(e) Interpretation – Phases of the Problem-Solving Agent:

- **Goal formulation:** from the current situation (both jugs empty) the agent adopts the goal "exactly 2 litres in jug X". This limits the behaviours to consider.
- **Problem formulation:** the agent decides which states and actions to consider, i.e. the (x, y) abstraction, the six actions, the goal test x = 2 and unit step cost (part b).
- **Search:** the agent looks through the state space (for example by BFS) for a sequence of actions leading from (0, 0) to a goal state, and finds the 6-action solution in part (d).
- **Execution:** the agent carries out the actions in order: Fill Y, Pour Y→X, Fill Y, Pour Y→X, Empty X, Pour Y→X.

Why can it execute without looking at new percepts? The environment is fully observable at the start (initial state known), deterministic (every action has exactly one predictable result), static (nothing changes except through the agent) and the model is known. With these four properties the agent can predict each state in advance, so new percepts would provide no additional information. This open-loop execution is possible only because of those properties. In a non-deterministic or partially observable environment the agent would have to monitor percepts during execution.

Question 3: Searching for Solutions – Uninformed Search Strategies:

Problem Understanding:

Start node = S; goal node = G. Edges (undirected, with cost): S–A 1, S–B 4, A–C 2, A–D 5, B–D 1, C–G 6, D–G 2. Neighbours in alphabetical order:

Node	S	A	B	C	D	G
Neighbours	A, B	C, D, S	D, S	A, G	A, B, G	C, D

Conventions: graph search with an explored set; the goal test is applied when a node is removed from the frontier; ties are broken alphabetically; BFS and DFS ignore edge costs while searching, but the path costs are computed using them.

(a) Breadth-First Search (queue, FIFO):

Initial queue: [S]. Explored set: { }.

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D]	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G]	{S, A, B, C, D}
6	G (goal ✓)	[ ]	{S, A, B, C, D, G}

Justification: Iteration 1 adds A and B. Iteration 2 adds C and D (S is explored). Iteration 3: B's neighbours D (already queued) and S (explored) are both skipped, so nothing is added. Iteration 4 adds G (reached from C). In iteration 5, D's neighbours A and B are explored and G is already queued. In iteration 6, G passes the goal test. G entered the queue from C, whose parent is A, whose parent is S.

BFS path: S → A → C → G; cost = 1 + 2 + 6 = 9.

(b) Depth-First Search (stack, top on the left):

Initial stack: [S]. Children are pushed so that the alphabetically first child is on top.

Iteration	Node Popped	Stack after Pushing Children
1	S	[A, B]
2	A	[C, D, B]
3	C	[G, D, B]

4	G (goal ✓)	[D, B]

Justification: After popping S, A is on top, so A is explored first. A's unexplored children are C and D, and C is pushed on top. C's unexplored child is G (A is explored). G is on top, popped next and passes the goal test, so the search stops with D and B still unexplored on the stack.

DFS path: S → A → C → G; cost = 9. Nodes popped = 4.

(c) Uniform-Cost Search (priority queue ordered by g(n)):

Entries are written Node:g(parent). Initial queue: [S:0].

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)] (D replaced: 6 → 5)
5	D	5	[G:7(D)] (G replaced: 9 → 7)
6	G (goal ✓)	7	[ ]

Justification: Iteration 2: from A, C costs 1 + 2 = 3 and D costs 1 + 5 = 6. Iteration 3: from C, G costs 3 + 6 = 9. Iteration 4: B (g = 4) gives D at 4 + 1 = 5, which is cheaper than 6, so the entry for D is replaced. Iteration 5: from D, G costs 5 + 2 = 7, cheaper than 9, so G is replaced. Iteration 6: G has the lowest g and is removed, passing the goal test. Since UCS removes nodes in order of g, the first goal removed is optimal.

UCS path: S → B → D → G; cost = 4 + 1 + 2 = 7.

(d) Comparison Table:

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS	S → A → C → G	9	6 (S, A, B, C, D, G)	No
DFS	S → A → C → G	9	4 (S, A, C, G)	No
UCS	S → B → D → G	7	6 (S, A, C, B, D, G)	Yes

(e) Interpretation:

UCS vs BFS: both removed 6 nodes, but they order the frontier differently. BFS expands by depth and ignores edge costs. Both S–A–C–G and S–B–D–G have 3 edges, so BFS returned the one reached first alphabetically (cost 9). UCS orders by path cost g(n), so it discovered the cheaper routes to D (5 instead of 6) and to G (7 instead of 9). UCS therefore found a cheaper path with the same number of removals.

DFS: DFS removed only 4 nodes because following the alphabetically first branch (A, then C) led straight to G, which is luck depending on the ordering. It cannot be trusted in general: it is not optimal (here it returned cost 9, not 7), it explores one deep branch before others and so can waste effort or follow very long or infinite paths, and its result depends on the order of expansion.

Explored set (BFS iteration 3): when B is dequeued, its neighbours are D and S. S is already explored and D is already in the queue (added in iteration 2). Without an explored set, S would be added again, and the search would loop S → A → B → S → …, re-expanding nodes and possibly never ending. The explored set (and the queue check) removes repeated states, so each node is expanded once.

Question 4: Search with Partial Information – Belief States:

Physical States (for reference):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Belief-State Formulation (Sensorless):

Component	Formulation
Belief states	Any subset of the 8 physical states S1–S8.
Initial belief state	{S1, S2, S3, S4, S5, S6, S7, S8}: the agent knows neither its location nor the dirt status, so all 8 states are possible.
Actions	Left, Right, Suck. The result is the set of results of the action on every state in the belief state: $b' = \{ \text{RESULT}(s, a) : s \in b \}$.
Goal test	Every state in the belief state is a goal state (both squares clean): $b \subseteq \{S7, S8\}$.
Path cost	1 per action (path cost = number of actions).

Justification: Since the agent cannot sense anything, the search is over sets of states. The goal must hold in all possible states, since the agent cannot tell which one it is actually in.

(b) Sensorless Action Sequence [Right, Suck, Left, Suck]:

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	{S2, S4, S6, S8}
2	Suck	{S4, S8}

3	Left	{S3, S7}
4	Suck	{S7}

Justification: Step 1 (Right): S1→S2, S2→S2, S3→S4, S4→S4, S5→S6, S6→S6, S7→S8, S8→S8, so the belief is {S2, S4, S6, S8}; the agent now surely is in the right square. **Step 2 (Suck):** S2 (R,D,D)→S4 (R,D,C); S4 stays S4 (right square is already clean); S6 (R,C,D)→S8; S8 stays S8, giving {S4, S8}. **Step 3 (Left):** S4→S3 (L,D,C), S8→S7 (L,C,C), giving {S3, S7}. **Step 4 (Suck):** S3→S7 (L,C,C), S7 stays S7, giving {S7}. The belief state is {S7}, a goal state: the agent has reached a goal state with certainty without any sensing.

(c) Percept-Based Belief-State Update:

Prediction: $\hat{b}$ = { RESULT(s, a) : s ∈ b }. Update: keep only the states in $\hat{b}$ that are consistent with the percept.

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	{S1, S3}
1	Suck	{S5, S7}	[Left, Clean]	{S5, S7}
2	Right	{S6, S8}	[Right, Dirty]	{S6}

Justification: Step 0: [Left, Dirty] is consistent only with states having the agent in L and a dirty left square: S1 (L,D,D) and S3 (L,D,C). **Step 1:** Suck cleans the left square: S1→S5 (L,C,D) and S3→S7 (L,C,C), so the prediction is {S5, S7}. [Left, Clean] holds in both (agent in L, left clean), so both stay. **Step 2:** Right moves the agent: S5→S6 (R,C,D), S7→S8 (R,C,C), giving {S6, S8}. [Right, Dirty] needs the right square to be dirty. S6 qualifies but S8 does not, so the belief shrinks to {S6}. The agent now knows exactly where it is and that only the right square needs cleaning (a Suck next reaches the goal).

(d) Number of Belief States:

A belief state is any subset of the 8 physical states, so there are 2^8 = **256** possible belief states (255 if the empty set is excluded).

Justification: The belief-state space grows exponentially (2^n) with the number of physical states n. Searching over physical states involves only 8 nodes, but searching over belief states involves up to 256, and each belief state may contain many physical states, so each node needs more memory and each step needs more computation. Plans must also work for all states in the belief state at once. For larger worlds this blow-up makes the problem much harder.

(e) Interpretation:

Why percepts shrink the belief state: a percept is evidence about the real state. After predicting what an action does, the agent discards every state that cannot produce the observed percept. In part (c) [Left, Dirty] cut 8 states to 2, and [Right, Dirty] cut {S6, S8} to {S6}. Each percept removes uncertainty, which is why the belief state becomes smaller. Note that [Left, Clean] did not shrink the belief because both S5 and S7 are consistent with it, so a percept only helps when it separates the possible states.

Difference from the sensorless case: in part (b) there is no percept update step. The belief changes only through the action's prediction (the sequence shrank from 8 states to {S7} only because Right and Suck are coercive actions that force different states into the same state, not because of observation). The agent must plan a fixed sequence that works for every possible state, and it ends up certain only about the result. With sensors in part (c), the agent alternates predict → update, so it can adapt its next action to what it perceives and ends with exact knowledge of the state ({S6}).

Why the sensorless plan is still rational: [Right, Suck, Left, Suck] reaches the goal state S7 from every one of the 8 possible initial states, so it guarantees success with no information at all and at the cost of only 4 actions. A rational agent picks the action sequence that maximises expected performance given what it knows. Under total uncertainty, a plan that is guaranteed to reach the goal is the best choice. Rationality does not require knowing the state; it requires acting sensibly with respect to the available knowledge.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Accuracy of Solution Path	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q. No. / Criteria	Understanding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Accuracy of Solution Path	Sub. Total (20)	Total %
1							
2							
3							
4							
Total							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____